

Guion de defensa individual — 10 minutos (formato reunión ejecutiva)

EFT ISY0101 · Agente Portuario EPV

El docente representa a la dirección de EPV. Tono: presentar una solución de negocio, no una tarea.

Minuto a minuto

0:00–1:00 — El problema (síntesis del caso)

Buenos días. Les presento la solución de asistencia normativa inteligente para EPV. El problema: la normativa operacional —seguridad, EPP, emergencias, convenio colectivo— está repartida en documentos extensos; los equipos pierden tiempo buscando y arriesgan errores de interpretación. Nuestra solución: un **agente de IA** que responde con la normativa oficial como fuente, evalúa cumplimiento, genera reportes y complementa con estándares internacionales — con trazabilidad completa de cada consulta.

1:00–2:30 — La solución en 3 fases (arquitectura)

Mostrar **diagrama de arquitectura** (1 lámina):

Lo construimos en 3 fases. Fase 1: un pipeline **RAG** — indexamos los 4 documentos oficiales en una base vectorial local (ChromaDB) con embeddings multilingües; el modelo responde con los fragmentos relevantes, no de memoria: eso reduce alucinaciones. Fase 2: lo convertimos en **agente ReAct** con gpt-4o-mini: razona, planifica y elige entre 4 herramientas — consulta de normativa, evaluación de cumplimiento, generación de reportes y búsqueda externa en Wikipedia para normativa internacional — con memoria de corto plazo (sesión) y largo plazo (entre sesiones). Fase 3: capa de **observabilidad y seguridad** sin tocar el agente: cada consulta queda registrada con latencias, tokens, relevancia y errores.

2:30–5:30 — DEMO en vivo (el corazón de la defensa)

Demo 1 — Agente (2 min): app EP2 abierta. Consulta preparada:

Consulta las normas de seguridad para trabajo nocturno y evalúa el riesgo de operadores sin casco en descarga nocturna

- Narrar el panel de razonamiento: "aquí se ve el patrón ReAct: **pensó** que necesita consultar primero, **ejecutó** el RAG, **observó** los fragmentos con su fuente, y encadenó la evaluación de cumplimiento — planificación multi-paso autónoma."

Demo 2 — Dashboard (1 min): dashboard EP3 abierto.

Cada consulta que vieron quedó registrada. Este dashboard muestra: latencia con su p95, desglose por fase — noten que el LLM concentra el 99% del tiempo —, consumo de tokens, tasa de errores por tipo y relevancia del retrieval separada por categoría de consulta.

5:30–7:30 — Resultados y la mejora implementada (datos, antes/después)

Tres hallazgos de la medición con 36 ejecuciones reales: **uno**, el cuello de botella es la generación LLM (99.6% del tiempo) — optimizar el RAG no paga. **Dos**, el score de similitud separa nítidamente consultas dentro y fuera del dominio (0.076 vs 0.039): definimos un umbral de 0.057 para detectar consultas que el corpus no cubre. **Tres**, bajo carga sostenida el free tier de la API rechazó un tercio de las peticiones (HTTP 429), inflando el p95 a 33.7 segundos.

>

Con ese diagnóstico **implementamos la mejora**: reintentos con backoff exponencial (max_retries=5) y timeout de 45s. Resultado medido en la re-evaluación con el mismo dataset: **la tasa de error de API cayó de**

33.3% a 5.6% — seis veces menos fallas — y la mediana bajó de 9.3 a 6.8 segundos. El costo: la cola p99 creció, porque los reintentos convierten errores rápidos en respuestas lentas pero exitosas — un trade-off correcto: en normativa de seguridad, una respuesta tardía vale más que un error. Este es el ciclo completo de observabilidad: medir → diagnosticar → mejorar → verificar.

7:30–9:00 — Seguridad y ética

En seguridad: sanitización de inputs contra prompt injection (OWASP LLM01) — en la evaluación bloqueó las 4 consultas adversariales antes de llegar al modelo, sin gastar tokens; anonimización de datos personales en logs alineada a la **Ley 19.628**; rate limiting local; y la API key por variable de entorno, nunca en el código. En lo ético: el agente cita sus fuentes, muestra su razonamiento, y se declara asistente — no reemplaza el juicio del prevencionista. Limitación declarada: los prompts viajan a la API; en producción evaluaríamos un modelo on-premise.

9:00–10:00 — Mejoras futuras y cierre

Próximas iteraciones, en orden de impacto: activar el umbral de relevancia con fallback automático; migrar a tier pago o modelo local; evaluación de calidad con LLM-as-judge; exportar métricas a Grafana con alertas sobre p95; y function calling nativo para eliminar los errores de parseo. En síntesis: entregamos un agente funcional, medido, seguro y con un ciclo de mejora demostrado con datos. Quedo atento a sus preguntas.

Checklist pre-demo (día de la defensa)

- [] ``.env`` con token vigente en ``EP3_observabilidad/`` (probar 1 consulta 30 min antes — ojo cuota diaria).
- [] Base ChromaDB de EP2 presente (``chroma_db/chroma.sqlite3``).
- [] Venv activado; ``streamlit run app.py`` (EP2, puerto 8501) y ``streamlit run dashboard.py`` (EP3, puerto 8502: usar ``--server.port 8502``).
- [] Pestañas abiertas ANTES de empezar: app agente, dashboard, diagrama de arquitectura.
- [] Consulta de demo ensayada (la de trabajo nocturno) — ensayar 2 veces: latencia real ~10-30s, llenar ese tiempo narrando el ReAct.
- [] Cronometrar el guion completo $\leq 9:30$ (margen).

Plan B si falla la API en vivo (rate limit / sin internet)

1. El **dashboard funciona sin API** (lee el JSONL local) → siempre demostrable.
2. Capturas de respaldo en ``EP3_observabilidad/docs/img/`` (dashboard_1..3.png) y el panel de razonamiento del agente — tener una corrida exitosa capturada en video o screenshots ANTES del día.
3. Frase preparada: "justamente este 429 que ven es el comportamiento que medimos y mitigamos con backoff — permítanme mostrar la corrida registrada" → pivotar al dashboard/capturas. Convierte el fallo en evidencia.

Los 10 números que no se te pueden olvidar

33.3% → error API antes · **5.6% → error API después (6× menos)** · 9.3→6.8s → mediana antes/después · 33.7s → p95 antes · 99.6% → LLM del tiempo · 24:1 → ratio tokens · 0.076/0.039 → similitud en/fuera · 0.057 → umbral · 4/4 → adversariales bloqueadas · 36 → ejecuciones del dataset.